

Introductory Guide to LLM Training and Attention Design

Pre-training, SFT, Post-RL Training, and Attention Metrics

Author / 作者: liangdacheng / 梁达成

Printable PDF version



Contents

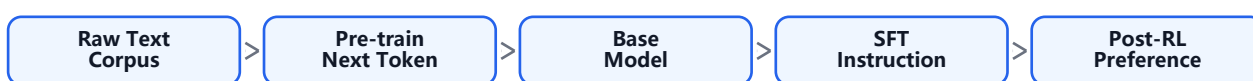
1. Learning Goals
2. Three Training Stages
3. Pre-training
4. Supervised Fine-Tuning
5. Post-RL Training: DPO, RLHF, GRPO
6. Transformer Core Structures
7. Proxy Metrics for Attention Design
8. Key Formulas
9. Recommended Experiment Protocol
10. Local Execution

1. Learning Goals

This handout merges two introductory notebooks into one printable guide: LLM training stages and attention-design metrics. The goal is to build intuition with small, inspectable experiments before scaling to real models, real datasets, and distributed training.

- Understand the goal, data format, and loss for pre-training, SFT, and post-RL training.
- Understand why embedding, attention, RoPE, residual paths, RMSNorm/LayerNorm, and MLP blocks exist.
- Use proxy metrics to evaluate attention designs before running full LLM training.
- Run the local PyTorch and Jupyter intro experiments from the provided conda environment.

2. Three Training Stages



Pre-training teaches the model a text distribution. SFT teaches the model to follow instructions and answer in the desired format. Post-RL training further aligns behavior with preferences, rewards, or verifiable objectives.

Stage	Data	Core loss / method	Output model
Pre-train	Large raw text corpora	causal LM next-token cross entropy	Base model
SFT	Instructions, questions, reference answers	assistant-token masked cross entropy	Instruction model
DPO	prompt + chosen + rejected	relative preference objective with reference model	Preference-aligned model
GRPO/RLHF	prompts + rewards/verifiers or preferences	policy optimization / reward model	Aligned model

3. Pre-training

The basic objective is next-token prediction. Inputs and labels are almost identical, but the loss is shifted by one token.

```
input:  x0 x1 x2 ... xT
target:   x1 x2 ... xT
loss:    CE(model(x_{<=t}), x_{t+1})
```

In real pre-training, the hard parts are usually not the formula. The hard parts are data quality, tokenization, throughput, stability, checkpointing, contamination control, and scale-out training. For learning, start with a tiny GPT below 100M parameters or a 0.5B-class continued pre-training run.

4. Supervised Fine-Tuning

SFT is still causal LM training, but labels should supervise only the assistant tokens. The user prompt is conditioning context, not a target to imitate.

```
input:  <bos>User: What is SFT?\nAssistant: SFT is...
labels: -100 -100 -100 ...           SFT is...
```

- The chat template must match the model and tokenizer.
- Watch truncation: it can remove the assistant tokens that carry the training signal.
- A small high-quality dataset often beats a large noisy one.
- Do not evaluate only training loss; also check format following, factuality, refusals, task suites, and generated samples.

5. Post-RL Training: DPO, RLHF, GRPO

Post-RL training is not meant to relearn language from scratch. It aligns output behavior with preferences, rules, rewards, or verifiable targets. DPO is the best entry point because it does not require a separate reward model.

$$L_{DPO} = -\log \text{sigmoid}(\beta * ((\log \pi_{\text{chosen}} - \log \pi_{\text{rejected}}) - (\log \pi_{\text{ref_chosen}} - \log \pi_{\text{ref_rejected}})))$$

Method	Required data	Best use case
DPO	chosen/rejected preference pairs	Intro alignment after SFT
Reward Model + PPO	preference pairs + RL sampling	Classic RLHF pipeline
GRPO	prompts + verifiable reward function	Math, code, and rule-verifiable tasks
RLAIF	AI feedback or rule-generated preferences	When human labels are scarce

6. Transformer Core Structures



Structure	Problem solved	Common issue if removed	Improvement directions
Embedding	Map discrete tokens to vectors	Token IDs carry no semantic distance	tokenizer, weight tying, domain vocabulary
Attention	Route context information	Weak long-range dependency handling	FlashAttention, GQA/MQA, sparse/window attention
RoPE/position encoding	Inject order and relative distance	Poor sequence order, brackets, and code position handling	YaRN, LongRoPE, ALiBi, xPos
Residual path	Information and gradient highway	Deep training becomes unstable	residual scaling, DeepNorm, gated residual
RMSNorm/LayerNorm	Stabilize activation distribution	loss spikes, exploding gradients, drift	Pre-Norm, QK-Norm, RMSNorm
MLP/SwiGLU	Nonlinear capacity and knowledge storage	Insufficient expressivity	SwiGLU, MoE, larger expansion ratio

7. Proxy Metrics for Attention Design

Before running full LLM training, proxy metrics can eliminate risky designs. No single metric replaces final loss, but the combination is useful.

Dimension	Metrics	Main prediction
-----------	---------	-----------------

Information flow	graph diameter, receptive field, rollout mass	Whether long-range information can travel
Stability	QK logits std/p99/p99.9, normalized entropy	Whether attention saturates or training breaks
Expressivity	effective rank, head diversity, ablation	Whether attention collapses or heads are wasted
Positional ability	retrieval vs distance, extrapolation ratio	Long-context and length extrapolation
Hardware	tokens/sec, TTFT, TPOT, MFU, KV cache	Practical training and serving viability
Small tasks	copy, key-value retrieval, multi-hop	Lower bound of structural capability

8. Key Formulas

Receptive field:

$$R_0(t) = \{t\}$$

$$R_l(t) = \text{union over } s \text{ in } \text{Attend}_l(t) \text{ of } R_{l-1}(s)$$

Attention logits:

$$z_{\{t,s\}} = q_t \cdot k_s / \sqrt{d_{\text{head}}}$$

$$p_{\{t,s\}} = \text{softmax}(z_{\{t,s\}})$$

Normalized entropy:

$$H_t = - \sum_s p_{\{t,s\}} \log p_{\{t,s\}}$$

$$H_{\text{norm}} = H_t / \log(N_t)$$

Effective rank:

$$p_i = \sigma_i / \sum_j \sigma_j$$

$$r_{\text{eff}} = \exp(- \sum_i p_i \log p_i)$$

KV cache:

$$\text{KV bytes} = \text{layers} * \text{batch} * \text{seq_len} * 2 * n_{\text{kv_heads}} * \text{head_dim} * \text{dtype_bytes}$$

9. Recommended Experiment Protocol

- First compute FLOPs, KV cache, and parameter count statically.
- On random inputs, inspect QK logits, entropy, effective rank, and head diversity.
- Train for 1k-5k steps and watch for loss spikes, NaN/Inf, and logit drift.
- Run copy, key-value retrieval, and multi-hop synthetic tasks.
- Measure retrieval vs distance, including lost-in-the-middle and length extrapolation.
- Only then run small-scale LLM pre-training loss and real task evaluation.

10. Local Execution

The working directory already contains the conda environment file, Jupyter startup script, and both notebooks.

```
cd /mnt/c/Users/Administrator/Desktop/llmintro
./verify_llm_intro_env.sh
./start_llm_intro_jupyter.sh
```

File	Purpose
llm_training_stages/llm_pretrain_sft_postrl_tutorial.ipynb	Pre-train, SFT, and DPO teaching experiment

<code>attention_design_metrics/attention_metrics_tutorial.ipynb</code>	Attention-design metric teaching experiment
<code>llm_training_stages/tiny_llm_training.py</code>	tiny GPT, SFT batch, DPO loss
<code>attention_design_metrics/attention_metrics_to_rch.py</code>	attention metric utilities
<code>environment.yml</code>	conda environment definition